

Tutorial 6 (31/8/2026)

1. A vehicle rental company manages Cars and Bikes. Create a Java program with a parent class `Vehicle` containing `vehicleNumber`, `brand`, `speed`, and `displayDetails()`. Derive `Car` and `Bike` from `Vehicle`, adding `numberOfDoors` and `hasGear` respectively. Override `displayDetails()` in both subclasses and demonstrate runtime polymorphism by using a `Vehicle` reference to invoke the appropriate overridden method.
2. An online shopping application supports Credit Card, UPI, and Net Banking payments. Create a Java program by declaring a `Payment` interface with a `pay(double amount)` method. Implement the interface in `CreditCardPayment`, `UPIPayment`, and `NetBankingPayment` classes with different implementations. Use a `Payment` reference to invoke `pay()` for different payment methods.
3. A mobile banking application provides basic login and logout features. An advanced version additionally supports fingerprint and face-recognition authentication. Create a Java program with a `BasicSecurity` interface and an `AdvancedSecurity` interface that extends it. Implement `AdvancedSecurity` in a `BankingApplication` class and demonstrate all the security operations.
4. A document management system stores different types of documents. Create a tagging interface `Confidential` and implement it in selected document classes. Use `instanceof` to identify whether a document is confidential.
5. Develop a Java program for a college management system by creating separate packages for Student and Course management. Create a `Student` class with student details and a `Course` class with course details. Place each class in its respective package, import them into a `Main` class, and display the student and course information.
6. Develop a Java-based Online Banking System that supports different types of accounts and payment methods. Create a parent class `Account` and derive `SavingsAccount` and `CurrentAccount` from it. Override the `displayDetails()` method in both subclasses and demonstrate runtime polymorphism using an `Account` reference.

Create a `Payment` interface with a `pay(double amount)` method and implement it in `UPIPayment` and `CardPayment`. Create another interface `SecurePayment` that extends `Payment` and adds a `verifyPayment()` method. Implement it in `UPIPayment`.

Create a tagging interface `OnlineTransaction` and use it to identify online payment classes. Organize the classes into suitable packages and create a `Main` class to demonstrate the complete system.